# FlagOS Open Computing Global Challenge S2 - Track 2

## MiniCPM5-2B inference throughput optimization on Iluvatar BI-V150 (v8)

Technical report - working draft, 2026-11-06

Registered team: [TO FILL - registered name]
Authors: [TO FILL - confirm attribution before submission]
Target hardware: 1 x Iluvatar BI-V150 (32 GiB)
Target model: MiniCPM5-2B
Implementation: v8, the consolidated v1 prefill + v5 decode optimization
Evaluation status: Self-run development results, not certified by the organizer.

### Abstract

We optimize MiniCPM5-2B inference throughput on the organizer-specified FlagOS / vLLM-FL stack by improving the existing Triton unified attention operator on BI-V150. The final selected v8 implementation combines (1) a guarded BF16 grouped-query-attention (GQA) prefill tiling change and (2) guarded physical-page lookup reuse in paged-KV pure decode. It preserves vLLM's attention metadata, KV-cache updates, backend interface, supported fallback configurations and official serving arguments. In the archived internal summary, v8 reached 2,331.29 total tokens/s in the 4K workload and 1,251.22 total tokens/s in the 16K workload, respectively +23.88% and +45.17% against our historical self-run baseline summaries. These results are preliminary until the raw final-v8 runs and full Level 3 accuracy evaluation are archived and reproduced.

### 1. Rules, goals and scope

The competition evaluates total tokens/s at two fixed scenarios and requires the same launch parameters as the baseline, accuracy >=0.95 on MATH-500 Level 3 (baseline 0.962), and latency safeguards (the competition describes a 1% tolerance around the baseline TTFT). The organizer also requires the specified vllm-plugin-FL branch (flagos-2026-s2) and FlagGems tag v5.3.5, submission of code/technical PDF/reproduction guide and a reproducible environment. The performance contribution is assessed on both the 4K and 16K workloads.

This implementation does not use hyperparameter-enabled quantization, speculative decoding, benchmark-script modifications, model changes, replacement of a major operator without optimizing it, deletion of the framework's operator-selection logic or bulk merge from a newer inference-framework branch. Improvements are in the optimized operator source, not request manipulation.

The project targets BI-V150 only. We do not claim results on MuXi C500, any other model, or untested device combinations.

### 2. Hardware and software

The inspected development environment snapshot from 2026-10-25 records BI-V150 with 32 GiB GPU memory, Ubuntu 24.04.2 LTS, Python 3.12.11, vLLM 0.24.0+empty, vllm-plugin-fl 0.3.0rc0+g13eb9be69, FlagGems 5.3.5, PyTorch 2.10.0+corex.4.5.0, flagtree 0.6.2a3+iluvatar3.6 and driver 4.5.0. Final source commit SHAs and software versions need to be recorded from the release host and matched to organizer pinning.

The model and dataset paths in the provided environment are /workspace/MiniCPM5-2B and /workspace/evalscope-datasets/math_500. No other model is involved.

### 3. Architecture and implementation

```
vllm serve -- fixed organizer flags
            |
     vllm-plugin-FL dispatch / FlagGems backend
            |  (preserve existing priority and fallback chain)
   Iluvatar BI-V150 selected attention backend
            |
     attention_biv150.py
     - inherited vLLM Triton attention metadata
     - inherited KV-cache update / attention interface
            |
     triton_unified_attention_biv150.py
            |
     +-------------------+----------------------+
     | eligible prefill  | eligible pure decode |
     |  BF16 GQA tiles   | paged-KV lookup reuse|
     +-------------------+----------------------+
            |
        original unified Triton operator with
        shape-gated compile-time specialization
            |
      normal fallback for unsupported shapes/features
```

Source entry points in the v8 tree:

- ‘vllm-plugin-FL/vllm_fl/dispatch/backends/flaggems/flaggems.py‘: chooses the BI-V150 specialized attention backend on Iluvatar, retains explicit FlagGems-attention priority when requested, and retains the other-vendor Triton fallback.
- ‘vllm-plugin-FL/vllm_fl/dispatch/backends/vendor/iluvatar/impl/attention_biv150.py‘: ‘BIV150TritonAttentionBackend‘ and ‘BIV150TritonAttentionImpl‘ reuse vLLM attention metadata and call the single unified attention launcher.
- ‘vllm-plugin-FL/vllm_fl/dispatch/backends/vendor/iluvatar/impl/triton_unified_attention_biv150.py‘: contains the actual existing-operator kernel improvements, including ‘optimize_prefill_gqa‘ and compile-time ‘DECODE_PAGE_LOOKUP‘.

No secondary v4/v6/v7 kernel is part of this selected v8 implementation.

## 4. Prefill: BF16 GQA tile reuse (v1 contribution)

MiniCPM5-2B’s eligible attention configuration uses 16 query heads, 2 KV heads and head dimension 128, so each KV head feeds 8 query heads. In the original launcher the corresponding initial BLOCK_M value is 16, giving BLOCK_Q = 16/8 = 2 query positions per KV group. The v8 optimize_prefill_gqa specialization sets BLOCK_M = 32 and BLOCK_Q = 32/8 = 4, allowing more query positions to reuse the same KV tile inside an attention block. This changes the query-side tiling strategy of the existing operator rather than substituting an unoptimized operator.

The gate requires max_seqlen_q > 1, 16 Q heads, 2 KV heads, head dimension 128, BF16 Q/K/V, no KV quantization, causal attention, and excludes incompatible features such as tensor descriptors, sliding window, alternative masking/bias, sink tokens, soft cap and output quantization. Noneligible cases retain original kernel launch geometry. The optimizer does not modify the number of output tokens or change model-level inference settings.

## 5. Decode: paged-KV physical-page lookup reuse (v5 contribution)

Decode repeatedly accesses the paged KV cache. A naive per-token lane address calculation can repeat identical block-table lookups for tokens residing on the same physical page. The v8 kernel’s compile-time DECODE_PAGE_LOOKUP branch indexes the physical page once per relevant KV tile/page and reuses that mapping across its token lanes while maintaining correct intra-page offsets and masking. The change is local to the address-generation operations inside the existing unified attention kernel, with no altered model attention semantics.

It is enabled only for pure decode (max_seqlen_q == 1), supported BF16 16Q/2KV/128D geometry, a positive

power-of-two block size aligned with the tile, nonquantized KV cache and compatible descriptor mode. All other cases use the original lookup path. Both the 2D and 3D softmax paths are considered through the existing runtime scheduling decisions; v8 does not force a 3D or 2D operator selection.

Independence: prefill requires max_seqlen_q > 1; decode lookup requires max_seqlen_q == 1. Both specializations cannot activate in the same invocation. This matters for mixed prefill/decode serving and protects unsupported shapes.

## 6. Experimental design

All throughput comparisons refer to the supplied benchmark's two scenarios:

```
Scenario   Prompt tokens   Generated tokens   Concurrency   Prompts
4K   4,096   1,024   64   256
16K   16,384   1,024   64   128
```

The official benchmark command is the unchanged benchmark_throughput_serve.py with --test-cases '[[4096,1024,64,256],[16384,1024,64,128]]'. The prescribed serving flags include cudagraph_mode=FULL_DECODE_ONLY, GPU memory utilization 0.85, model alias minicpm and maximum model length 131072. Accuracy is tested by the official EvalScope command on MATH-500 Level 3 (105 items) with temperature 1.0, top_p 0.95 and max_tokens 32768.

For valid comparisons, pair full successful workloads on the same image, driver, model weights, GPU and server parameters, and record background load, KV pressure, prefix-cache state and any request failures. The archived baseline averages and v8 summary came from development sessions; they have not yet been assembled as a matched, reproducible final-run evidence bundle.

## 7. Preliminary throughput results

```
Workload   Internal baseline total tok/s   v8 total tok/s   Absolute gain   Relative gain
4K   1,881.95   2,331.29   +449.34   **+23.88%**
16K   861.88   1,251.22   +389.34   **+45.17%**
```

Calculation: gain = (v8_total_tok_s / baseline_total_tok_s - 1) * 100%, rounded to 2 decimal places. These historical baseline values are the summary averages reported in 4k16K_summary_benchmark.csv (first archived baseline series); v8 is the full-run value in profiling/v8/summary. Neither is a substitute for organizer verification.

```
Workload   Internal baseline mean TTFT (ms)   v8 mean TTFT (ms)   Direction
4K   12,309.34   8,738.76   lower in this comparison
16K   636,512.47   428,970.85   lower in this comparison
```

The preliminary TTFT comparison points in the right direction, but should not be interpreted as proof of meeting the event's latency guardrail without a same-conditions final baseline. The 16K values are large because high-concurrency long-context serving stresses the memory and scheduler; see Section 9.

Evidence limits: the repository archives the historical baseline raw runs and a written final-v8 summary, but does not yet archive the original final-v8 raw full-run CSV and server trace. It would be incorrect to claim confirmed organizer throughput or per-run statistical significance. The displayed gains are self-run estimates only.

## 8. Accuracy and functionality

The competition's formal accuracy threshold is 0.95, against a baseline of 0.962. The v8 profiling summary reports 98.1% accuracy, and an older v1 Level 3 run documents 98.1% on 105 samples. The original v8-matched complete EvalScope report is not currently attached to the draft; a separate earlier baseline log also records a 97.1% run. Therefore 98.1% is a reported development result, not independently substantiated here for final v8.

The v8 repository contains targeted functional tests:

- 'tests/functional_tests/test_biv150_attention.py': prefill/backend and fallback tests.

- ‘tests/functional_tests/test_biv150_decode.py‘: decode/page-table specialization tests.

Development logs show these suites have run successfully in the project history. The final submission must include fresh exit statuses, complete logs, successful 105/105 EvalScope outcomes and result JSON, plus proof that the serving process imported the intended implementation. Functional unit passes alone do not prove MATH-500 accuracy.

## 9. Bottlenecks, alternative experiments and lessons

Long-context throughput is not only a function of kernel computation time. Previous v5 diagnostics estimated a 42 KiB per-token KV state for the model, reported a server KV budget around 522,960 tokens (~20.95 GiB), and observed near-saturated cache usage, queued requests and preemptions. Holding 64 independent 16,384-token prefixes requires 1,048,576 cache tokens, more than that observed budget. Consequently, scheduler waiting, cache residency and prefix reuse can dominate mean TTFT. Optimizing decode page lookup reduces per-kernel address work but does not eliminate the KV-capacity constraint.

The project tested other stack candidates in half-run development comparisons. The surviving v8 combination v1 + v5 yielded a recorded 4K/16K half-run of 2,128.76/1,238.72 total tok/s; the exploratory v1+v3 and v1+v4 versions did not show consistent 16K gains, and additions of v6 KV tiling or v7 grouped GEMM did not give clear serving-level benefits over v1+v5. These were screening experiments, not equivalent full official benchmarks. They are excluded from the final solution to avoid complexity without supported benefit.

The wider performance analysis also identified potential CPU/GPU launch gaps, preemption and KV data movement, as well as differing cache behavior between 4K and 16K. No new scheduler policy or benchmark modification is claimed as part of v8.

## 10. Reproduction and compliance

Follow the submitted readme.md to install the two supplied source trees from the archive into the official environment, run the functional tests, launch vllm serve with exact required parameters, execute the MATH-500 Level 3 EvalScope command, and run the unmodified official 4K/16K benchmark.

The delivered ZIP must contain:

```
<registered-team-name>/
  FlagGems/
  vllm-plugin-FL/
  readme.md
  report.pdf
```

Any final result table should identify source Git SHAs, software versions, exact log filenames, benchmark success counts and all error/timeout events. The judging score is based on organizer reproduction, not the internal measurements above.

## 11. Innovation and generality

The primary technical contribution is a phase-aware and shape-safe improvement to one production attention operator: query-side reuse in supported GQA prefill, and page-table reuse within tiles during supported paged-KV decode. Both changes are compile-time gated, independently measurable, and retain baseline behavior outside their validity region. The implementation reuses vLLM’s metadata and KV-cache life cycle instead of introducing duplicate attention APIs or deleting dispatch logic. This separation of prefill and decode bottlenecks is broadly applicable to paged-attention workloads, although speedups on other models or chips are not asserted.

## 12. Open items before final submission

1. Record and verify the exact framework branch and FlagGems source revisions in the packaged v8

snapshots.
2. Attach original full v8 benchmark raw CSVs, serving logs, test command transcript, GPU metrics and success counts; perform a matched baseline rerun if possible.
3. Archive the final v8 MATH-500 Level 3 105-sample report and error inspection, and verify accuracy >=0.95.
4. Verify actual engine-process kernel selection, and check TTFT against official guardrails.
5. Replace author/team placeholders, add reviewed figures/plots generated from the raw run files, and proofread the exported PDF.
6. Reproduce the complete archive in a clean official image and submit by the deadline.

## Sources and traceability

- Competition rules: https://flagos.io/race-detail-season2?id=539vlt2p&lang=cn
- v8 implementation: ‘Co_Kaggle/g7/tests_BI-V150/Optimization/v8/‘
- v8 source description: ‘Optimization/v8/readme.md‘
- Internal v8 results: ‘profiling/v8/summary‘
- Internal baseline: ‘4k16K_rawrun_benchmark.csv‘ and ‘4k16K_summary_benchmark.csv‘
- v1 Level 3 EvalScope evidence: ‘profiling/v1/summary.txt‘
- v5 bottleneck analysis: ‘profiling/v5/v5_decode_16k_bottleneck_findings.md‘
- Environment snapshots: ‘HW_SW_info.txt‘ and ‘FlagOS_Env.txt‘

Disclosure: This report is a draft for transparent technical review, not an assertion that the official submission has been created, tested, uploaded or accepted.